Part 2 – Data Build Tools Demo

1. Pertama buka aplikasi visual studio codenya, kemudian open folder dengan nama DBT-DEMO.

2. Lalu buat file docker-compose.yml sesuai dari repository yang ada.

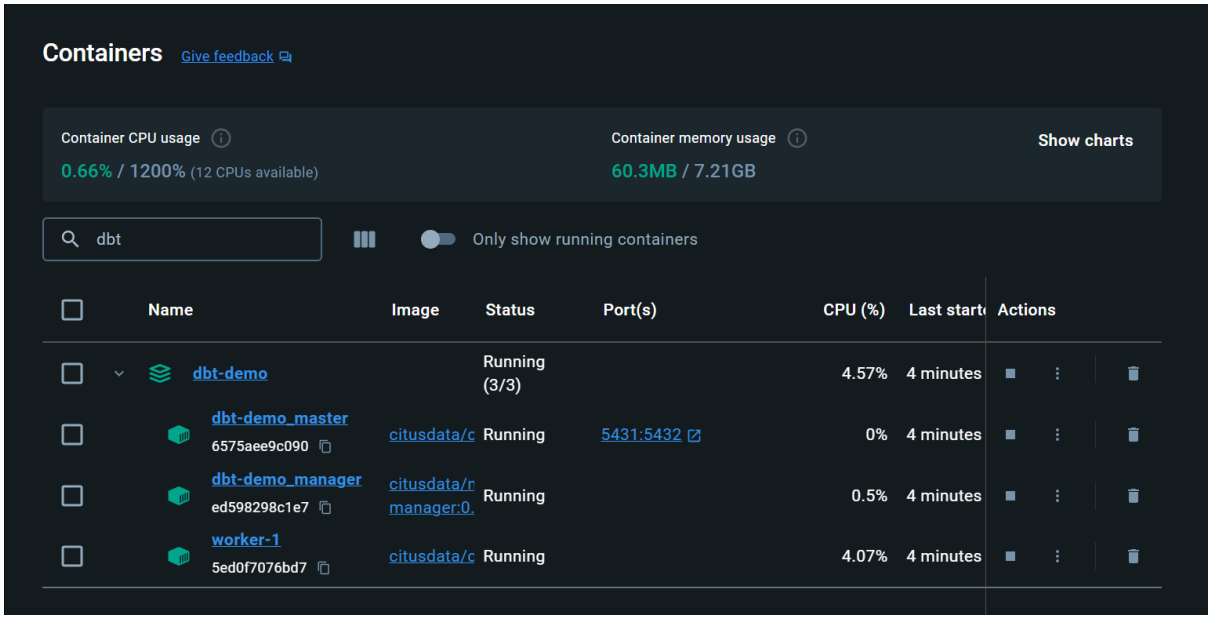
```yaml
docker-compose.yml
version: '3'

services:

  master:
    container_name: "${COMPOSE_PROJECT_NAME:-citus}_master"
    image: "citusdata/citus:12.0.0"
    ports:
      - "${COORDINATOR_EXTERNAL_PORT:-5431}:5432"
    labels:
      - "com.citusdata.role=Master"
    environment: &AUTH
      - POSTGRES_PASSWORD=pass
      - POSTGRES_USER=postgres
      - POSTGRES_DB=store
      - PGUSER=postgres
      - POSTGRES_HOST_AUTH_METHOD=trust
    networks:
      - postgres-network
    volumes:
      - ./citus-db-data/:/var/lib/postgresql/data/
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

  worker:
    image: "citusdata/citus:12.0.0"
    labels:
      - "com.citusdata.role=Worker"
    depends_on:
      - manager
    environment: *AUTH
    command: "/wait-for-manager.sh"
    volumes:
      - ./citus-healthcheck:/healthcheck
```

Contoh Sebagian tampilan dari isi docker-compose.yml yang baru saja dibuat.

3. Jalankan docker dengan mengetik “docker compose up -d” pada terminal.

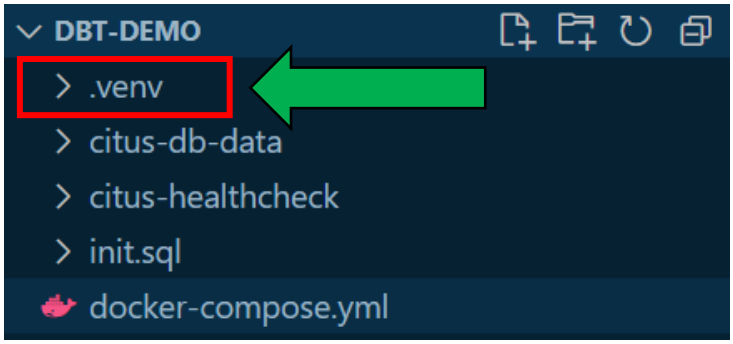
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                                   bash

tawhe@Ibnu_Emil MINGW64 ~/Documents/DBT-DEMO (main)
$ docker compose up -d
[+] Running 5/5
 ✓ Network dbt-demo_default           Created                                                    0.1s
 ✓ Network dbt-demo_postgres-network  Created                                                    0.1s
 ✓ Container dbt-demo_master          Started                                                    0.8s
 ✓ Container dbt-demo_manager         Started                                                    1.1s
 ✓ Container dbt-demo-worker-1        Started                                                    1.4s
```

Berikut merupakan tampilan perintah “docker compose up -d” sudah berjalan.



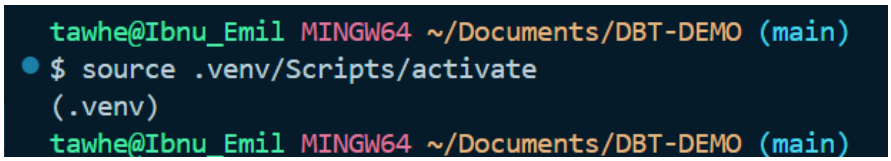
Container berhasil dibuat pada aplikasi docker.

4. Buat environment dengan mengetik “pyhton -m venv .venv”.



Environment sudah berhasil dibuat.

5. Aktifkan environment yang sudah dibuat sebelumnya dengan perintah “source .venv/Scripts/activate”.



6. Install adapter dengan perintah “pip install dbt-postgres”.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

  Using cached pydantic_core-2.20.1-cp312-none-win_amd64.whl.metadata (6.7 kB)
Downloading dbt_postgres-1.8.2-py3-none-any.whl (33 kB)
Downloading dbt_adapters-1.4.0-py3-none-any.whl (159 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 159.9/159.9 kB 737.2 kB/s eta 0:00:00
Downloading dbt_common-1.7.0-py3-none-any.whl (78 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 78.1/78.1 kB 1.1 MB/s eta 0:00:00
Downloading agate-1.9.1-py2.py3-none-any.whl (95 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 95.1/95.1 kB 1.1 MB/s eta 0:00:00
Downloading dbt_core-1.8.5-py3-none-any.whl (900 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 900.2/900.2 kB 280.6 kB/s eta 0:00:00
Using cached psycopg2_binary-2.9.9-cp312-cp312-win_amd64.whl (1.2 MB)
Downloading Babel-2.15.0-py3-none-any.whl (9.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.6/9.6 MB 293.8 kB/s eta 0:00:00
Using cached click-8.1.7-py3-none-any.whl (97 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading dbt_extractor-0.5.1-cp38-abi3-win_amd64.whl (283 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 283.5/283.5 kB 277.7 kB/s eta 0:00:00
Downloading dbt_semantic_interfaces-0.5.1-py3-none-any.whl (119 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 119.7/119.7 kB 319.0 kB/s eta 0:00:00
Downloading deepdiff-7.0.1-py3-none-any.whl (80 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 80.8/80.8 kB 205.2 kB/s eta 0:00:00
Downloading isodate-0.6.1-py2.py3-none-any.whl (41 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 41.7/41.7 kB 118.8 kB/s eta 0:00:00
```

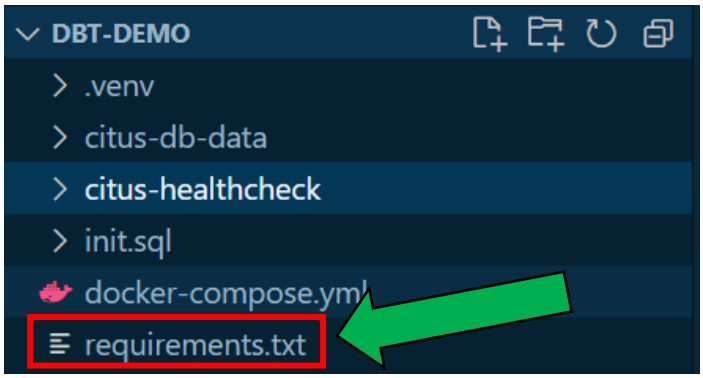
Berikut adalah sebagian tampilan dari proses yang sedang berjalan dari perintah “pip install dbt-postgres”.

7. Lalu jalankan perintah untuk packages yang akan diinstall dengan perintah “pip freeze | grep dbt”.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

tawhe@Ibnu_Emil MINGW64 ~/Documents/DBT-DEMO (main)
$ pip freeze | grep dbt
dbt-adapters==1.4.0
dbt-common==1.7.0
dbt-core==1.8.5
dbt-extractor==0.5.1
dbt-postgres==1.8.2
dbt-semantic-interfaces==0.5.1
(.venv)
tawhe@Ibnu_Emil MINGW64 ~/Documents/DBT-DEMO (main)
```

Berikut ini tampilan dari hasil perintah sebelumnya.

8. Setelah itu jalankan perintah “pip freeze | grep dbt >> requirements.txt” untuk memasukkan packages ke dalam file requirements.txt.



File “requirements.txt” sudah berhasil dibuat.

9. Lalu jalankan file tersebut dengan perintah “pip install -r requirements.txt”

10. Lakukan inisiasi kepada my_project dengan perintah “dbt init my_project” yang nanti akan menjadi folder utama dari dbt tersebut.

11. Buat folder “dbt-profiles” lagi dengan perintah “mkdir dbt-profiles”.

12. Di dalam folder “dbt-profiles” buat sebuah file dengan nama “profiles.yml” dengan perintah “touch dbt-profiles/profiles.yml”.

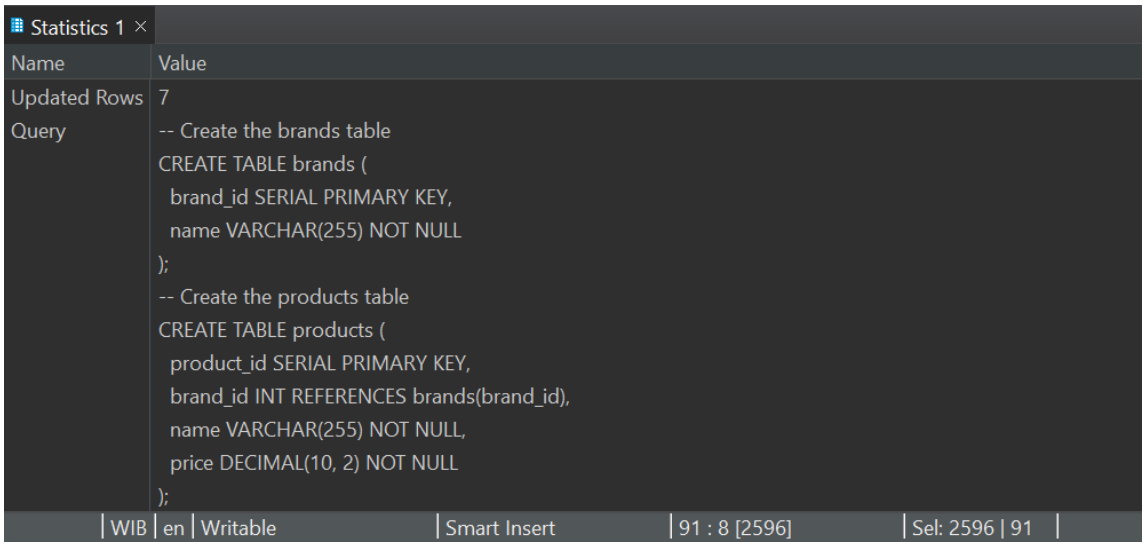
13. Setelah itu jalankan perintah “export DBT_PROFILES_DIR=$(pwd)/dbt-profiles”.

14. Kemudian isi file pada “profiles.yml” sesuai yang ada di repository.
    Berikut ini adalah isi dari file “profiles.yml” yang sesuai dengan repository.

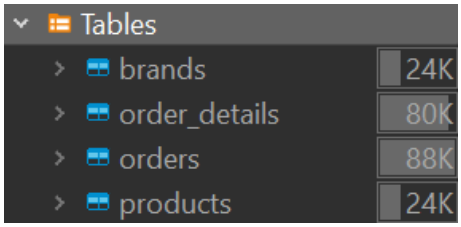
dbt-profiles > ! profiles.yml

```yml
my_project:
  outputs:

    dev:
      type: postgres
      threads: 1
      host: localhost
      port: 5431
      user: postgres
      pass: pass
      dbname: store
      schema: public

  target: dev
```

15. Kemudian buka aplikasi DBeaver dan pilih connection dbname store dengan port 5431, lalu pilih Databases > store > Schemas > public > Tables. Pada Table buat quary sesuai dengan repository dan jalankan quary tersebut.

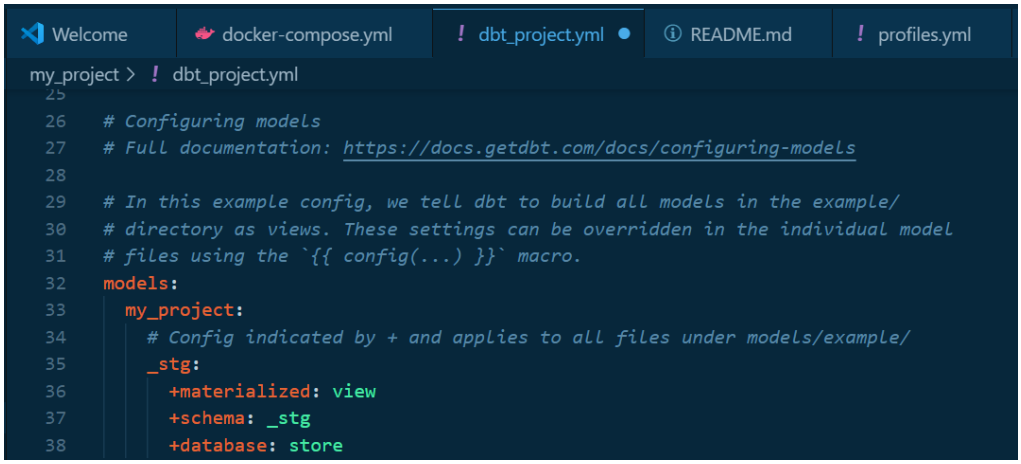


Berikut ini adalah hasil dari eksekusi quary yang sudah dibuat sebelumnya, dan eksekusi berhasil.



Tabel sekarang sudah berhasil dibuat.

16. Set “dbt_project.yml” seperti gambar berikut ini :



17. Rubah nama “models/example” menjadi models/_stg agar sesuai dengan isi file dbt_project.yml.

18. Rubah nama “schema.yml” menjadi “raw_schema.yml” dan juga rubah isi dari file tersebut sesuai dengan repository.

19. Untuk membuat model, buat file “stg_schema.yml” pada folder “models/_stg” dan isilah sesuai repository seperti berikut ini :

my_project > models > _stg > ! stg_schema.yml

```yaml
version: 2

models:
  - name: daily_sales
    description: "Aggregated sales metrics per day"
    columns:
      - name: order_date
        description: "The date of the orders"
        tests:
          - not_null
      - name: total_quantity
        description: "Total quantity of products sold"
        tests:
          - not_null
      - name: total_revenue
        description: "Total revenue for the day"
        tests:
          - not_null
```

20. Pada file dengan nama “my_first_dbt_model.sql” dirubah menjadi “stg_brands.sql”, lalu buatlah quary seperti berikut ini, karena ini format yang akan dipakai jika kita pertama kali mengambil dari raw :

my_project > models > _stg > stg_brands.sql

```sql
select
    brand_id::int as brand_id
    , name as brand_name
from {{ source('store', 'brands') }}
```

21. Pada file dengan nama “my_second_dbt_model.sql” dirubah menjadi “stg_products.sql”, lalu buatlah quary seperti berikut ini :

my_project > models > _stg > stg_products.sql

```sql
select
    product_id::int as product_id
    , brand_id::int as brand_id
    , name as product_name
    , price as product_price
from {{ source('store', 'products') }}
```

22. Buat file dengan nama “stg_orders.sql”, lalu buatlah quary seperti berikut ini :

my_project > models > _stg > stg_orders.sql

```
select
    order_id::int as order_id
    , order_date::timestamp as order_at
    , customer_phone
from {{ source('store', 'orders') }}
```

23. Buat file dengan nama “stg_order_details.sql”, lalu buatlah quary seperti berikut ini :

my_project > models > _stg > stg_order_details.sql

```
select
    order_details_id::int as order_details_id
    , order_id::int as order_id
    , product_id::int as product_id
    , quantity as order_qty
    , price as unit_sales
from {{ source('store', 'order_details') }}
```

24. Buatlah quary pada file “stg_schema.yml” seperti berikut ini :

my_project > models > _stg > ! stg_schema.yml

```
version: 2

models:
  - name: stg_brands
    description: "Brand staging"
    columns:
      - name: brand_id
        tests:
          - not_null
          - unique

  - name: stg_products
    description: "Product staging"
    columns:
      - name: product_id
        test:
          - not_null
          - unique

      - name: brand_id
        test:
          - not_null
```

my_project > models > _stg > ! stg_schema.yml

```
models:
  - name: stg_orders
    description: "Order staging"
    columns:
      - name: order_id
        test:
          - not_null
          - unique

  - name: stg_order_details
    description: "Order details staging"
    columns:
      - name: order_detail_id
        test:
          - not_null
          - unique
```

25. Buatlah quary pada file “dbt_project.yml” seperti berikut ini :

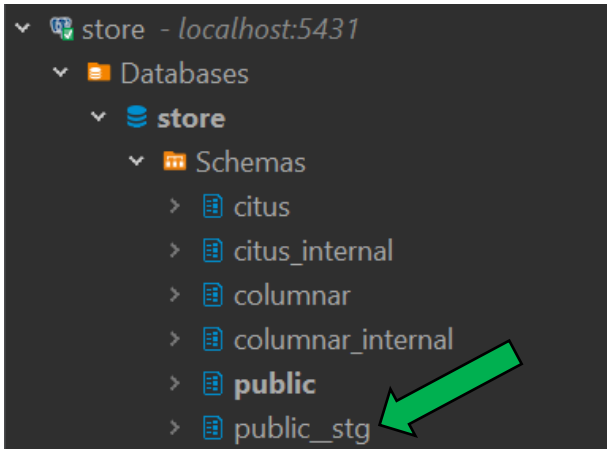
```
my_project > ! dbt_project.yml
32  models:
33    my_project:
34      # Config indicated by + and applies to all files under models/example/
35      _stg:
36        +materialized: view
37        +schema: _stg
38        +database: store
39      -int:
40        +materialized: table
41        +schema: _int
42        +database: store
```

26. Lalu masuk ke folder “my_project” dengan perintah “cd my_project”.

27. Kemudian jalankan dbt dengan perintah “dbt run”.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS        bash - my_project

16:47:13  1 of 4 START sql table model public__stg.stg_brands ............................ [RUN]
16:47:13  1 of 4 OK created sql table model public__stg.stg_brands ....................... [SELECT 3 in 0.16s]
16:47:13  2 of 4 START sql view model public__stg.stg_order_details ...................... [RUN]
16:47:13  2 of 4 OK created sql view model public__stg.stg_order_details ................. [CREATE VIEW in 0.09s]
16:47:13  3 of 4 START sql view model public__stg.stg_orders ............................ [RUN]
16:47:13  3 of 4 OK created sql view model public__stg.stg_orders ....................... [CREATE VIEW in 0.08s]
16:47:13  4 of 4 START sql view model public__stg.stg_products ......................... [RUN]
16:47:13  4 of 4 OK created sql view model public__stg.stg_products ..................... [CREATE VIEW in 0.08s]
16:47:13
16:47:13  Finished running 1 table model, 3 view models in 0 hours 0 minutes and 0.75 seconds (0.75s).
16:47:13
16:47:13  Completed successfully
16:47:13
16:47:13  Done. PASS=4 WARN=0 ERROR=0 SKIP=0 TOTAL=4
(.venv)
tawhe@Ibnu_Emil MINGW64 ~/Documents/DBT-DEMO/my_project (main)
$
```

DBT berjalan dengan sesuai harapan, semua sukses, ditandai dengan tulisan **PASS=4, WARN=0, ERROR=0, SKIP=0, TOTAL=0** dan juga tulisan “Completed successfully “



Pada aplikasi DBeaver juga dapat dibuktikan bahwa schema public__stg berhasil dibuat.

28. Langkah terakhir adalah dengan menguji apakah DBT yang sudah dibuat, sesuai dengan yang diharapkan. Untuk itu lakukan perintah “dbt test” pada terminal.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                    bash - my_project

17:05:46  1 of 19 START test not_null_stg_brands_brand_id ............................... [RUN]
17:05:46  1 of 19 PASS not_null_stg_brands_brand_id ..................................... [PASS in 0.06s]
17:05:46  2 of 19 START test source_not_null_store_brands_brand_id ...................... [RUN]
17:05:46  2 of 19 PASS source_not_null_store_brands_brand_id ........................... [PASS in 0.06s]
17:05:46  3 of 19 START test source_not_null_store_brands_name ........................... [RUN]
17:05:46  3 of 19 PASS source_not_null_store_brands_name ................................ [PASS in 0.06s]
17:05:46  4 of 19 START test source_not_null_store_order_details_order_detail_id ......... [RUN]
17:05:46  4 of 19 PASS source_not_null_store_order_details_order_detail_id ............... [PASS in 0.06s]
17:05:46  5 of 19 START test source_not_null_store_order_details_price ................... [RUN]
17:05:46  5 of 19 PASS source_not_null_store_order_details_price ......................... [PASS in 0.06s]
17:05:46  6 of 19 START test source_not_null_store_order_details_quantity ............... [RUN]
17:05:47  6 of 19 PASS source_not_null_store_order_details_quantity ...................... [PASS in 0.06s]
17:05:47  7 of 19 START test source_not_null_store_orders_order_date ..................... [RUN]
17:05:47  7 of 19 PASS source_not_null_store_orders_order_date ........................... [PASS in 0.04s]
17:05:47  8 of 19 START test source_not_null_store_orders_order_id ...................... [RUN]
17:05:47  8 of 19 PASS source_not_null_store_orders_order_id ............................ [PASS in 0.04s]
17:05:47  9 of 19 START test source_not_null_store_products_name ........................ [RUN]
17:05:47  9 of 19 PASS source_not_null_store_products_name ............................... [PASS in 0.05s]
17:05:47  10 of 19 START test source_not_null_store_products_price ...................... [RUN]
17:05:47  10 of 19 PASS source_not_null_store_products_price ............................ [PASS in 0.04s]
17:05:47  11 of 19 START test source_not_null_store_products_product_id .................. [RUN]
17:05:47  11 of 19 PASS source_not_null_store_products_product_id ........................ [PASS in 0.06s]
17:05:47  12 of 19 START test source_relationships_store_order_details_order_id__order_id__source_store_orders_  [RUN]
17:05:47  12 of 19 PASS source_relationships_store_order_details_order_id__order_id__source_store_orders_  [PASS in 0.06s]
17:05:47  13 of 19 START test source_relationships_store_order_details_product_id__product_id__source_store_products_  [RUN
]
17:05:47  13 of 19 PASS source_relationships_store_order_details_product_id__product_id__source_store_products_  [PASS in 0
.06s]
17:05:47  14 of 19 START test source_relationships_store_products_brand_id__brand_id__source_store_brands_  [RUN]
17:05:47  14 of 19 PASS source_relationships_store_products_brand_id__brand_id__source_store_brands_  [PASS in 0.05s]
17:05:47  15 of 19 START test source_unique_store_brands_brand_id ....................... [RUN]
17:05:47  15 of 19 PASS source_unique_store_brands_brand_id ............................ [PASS in 0.07s]
17:05:47  16 of 19 START test source_unique_store_order_details_order_detail_id ......... [RUN]
17:05:47  16 of 19 PASS source_unique_store_order_details_order_detail_id ............... [PASS in 0.06s]
17:05:47  17 of 19 START test source_unique_store_orders_order_id ....................... [RUN]
17:05:47  17 of 19 PASS source_unique_store_orders_order_id ............................ [PASS in 0.05s]
17:05:47  18 of 19 START test source_unique_store_products_product_id ................... [RUN]
17:05:47  18 of 19 PASS source_unique_store_products_product_id ......................... [PASS in 0.07s]
17:05:47  19 of 19 START test unique_stg_brands_brand_id ................................ [RUN]
17:05:47  19 of 19 PASS unique_stg_brands_brand_id ..................................... [PASS in 0.04s]
17:05:47
17:05:47  Finished running 19 data tests in 0 hours 0 minutes and 1.34 seconds (1.34s).
17:05:47
17:05:47  Completed successfully
17:05:47
17:05:47  Done. PASS=19 WARN=0 ERROR=0 SKIP=0 TOTAL=19
(.venv)
tawhe@Ibnu_Emil MINGW64 ~/Documents/DBT-DEMO/my_project (main)
$
```

Dari hasil diatas dapat disimpulkan bahwa hasil dari perintah “dbt test” berhasil sesuai dengan yang diinginkan, ditandai dengan hasil **PASS=19, WARN=0, ERROR=0, SKIP=0, TOTAL=19** dan juga tulisan “Completed successfully “